Práctica de la Unidad II

Estructuras de datos básicas

Versión WEB / C# / WebAssembly

Institución: Universidad Tecnológica Metropolitana

Asignatura: Estructura de Datos

Docente: Ing. Mirian Magaly Canché Caamal

Integrantes: Pendiente de completar por el equipo

Grupo: Pendiente de completar

Entrega: 23 de octubre de 2026

Web pública: https://victormanuelxd360-afk.github.io/Practica_U2_ED032026/

Doce ejercicios de listas, pilas y colas ejecutados mediante modelos C# compartidos. Las capturas se generaron desde los controles reales de la versión pública en el navegador. Los nombres completos de integrantes y grupo deben completarse antes de entregar.

Ejecución y arquitectura

La web pública funciona en GitHub Pages y ejecuta los algoritmos en C# WebAssembly. No depende de una computadora encendida. WebPublica contiene el motor para navegador; CSharp/Modelos.cs contiene las estructuras; Web/Estado.cs coordina los ejercicios. Web conserva la alternativa ASP.NET Core con sesiones de servidor.

El ZIP incluye código, solución, pruebas, capturas y Publicacion, una copia local para Windows x64 que no necesita instalar .NET. INICIAR_LOCAL.cmd abre esa copia. ABRIR_EN_LINEA.url abre la web pública. Para modificar el código se necesita SDK .NET 10.

L1. Productos disponibles y retirados

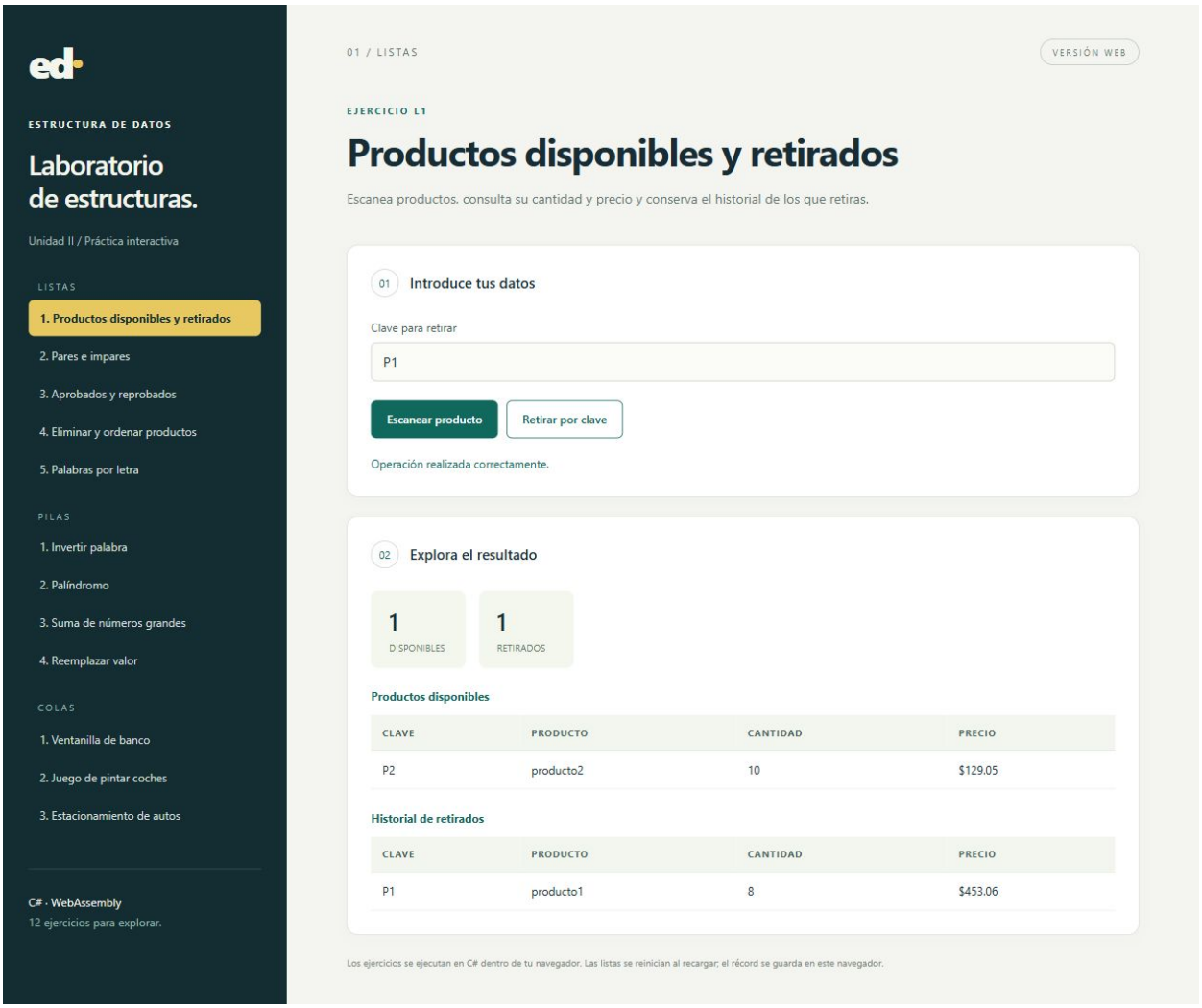
Problema: Administrar el almacén de productos escaneados y conservar los retirados.

Entradas: Clave del producto a retirar; la cantidad y el precio se generan automáticamente.

Proceso: Escanear crea un objeto Producto y lo agrega a List<Producto>. Retirar busca por clave y mueve el objeto a otra lista.

Salidas: Productos disponibles y retirados, con cantidades, precios y número de elementos.

Funcionamiento y evidencia: Pulsa Escanear producto dos veces; escribe P1 y pulsa Retirar producto. La imagen muestra el producto1 retirado y el producto2 disponible. Una clave inexistente produce un mensaje sin perder datos.



L2. Pares e impares

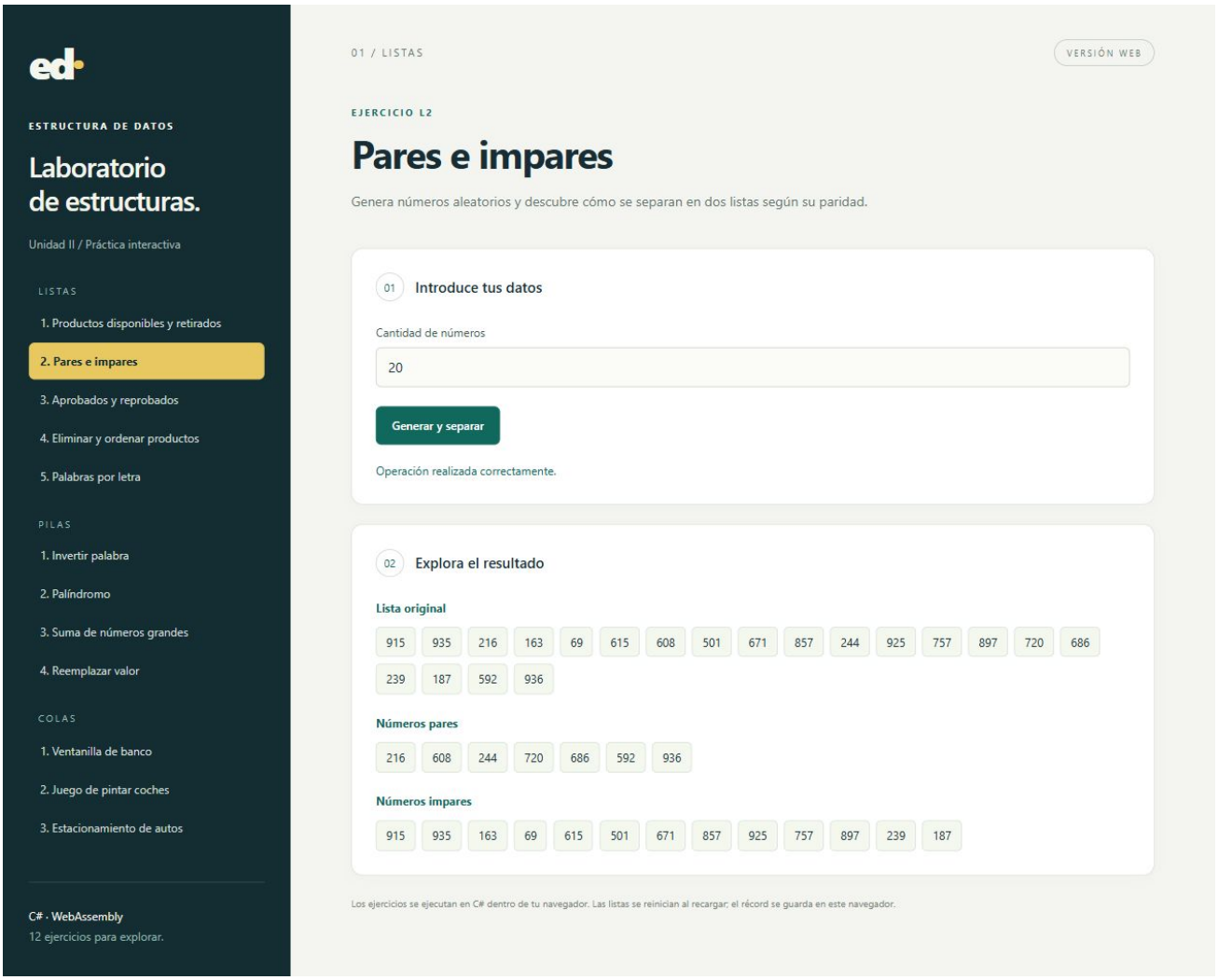
Problema: Separar una lista de números aleatorios en listas por paridad.

Entradas: Cantidad entera entre 1 y 500.

Proceso: Genera números de 0 a 999 y usa el residuo entre dos para agregarlos a las listas de pares e impares.

Salidas: Lista original y las dos listas clasificadas.

Funcionamiento y evidencia: Introduce 20 y pulsa Generar y separar. La captura muestra las tres listas; ningún número se pierde. Cero, decimales y letras se rechazan como cantidad.



L3. Aprobados y reprobados

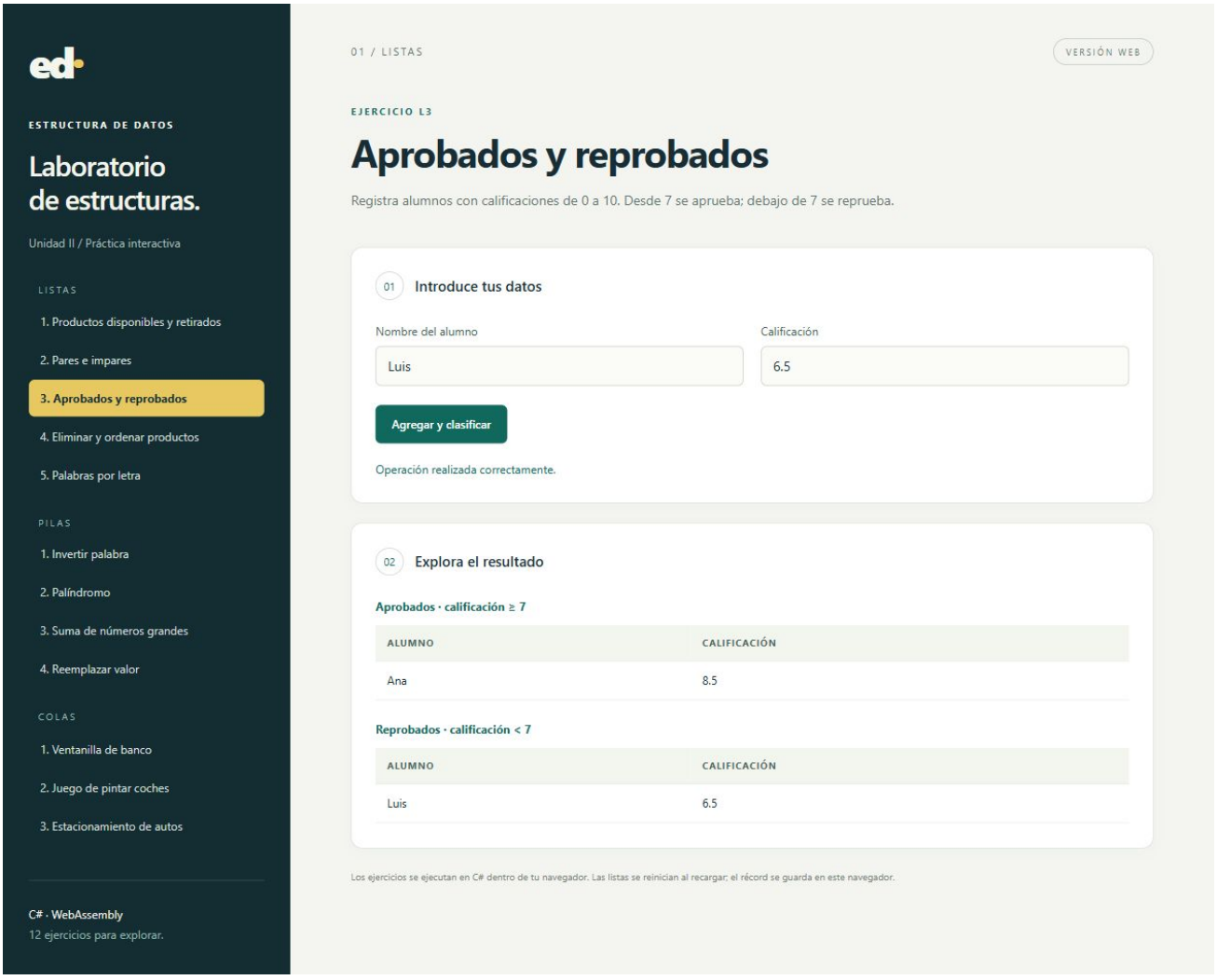
Problema: Clasificar alumnos según su calificación.

Entradas: Nombre y calificación decimal de 0 a 10.

Proceso: Agrega al alumno y extrae de la lista inicial los que tengan calificación menor que siete, trasladándolos a reprobados.

Salidas: Listas de aprobados y reprobados con sus notas.

Funcionamiento y evidencia: Registra Ana con 8.5 y Luis con 6.5. La captura muestra a cada alumno en su lista. Una nota igual a 7 aprueba; un nombre vacío o nota fuera de rango genera error.



L4. Eliminar y ordenar productos

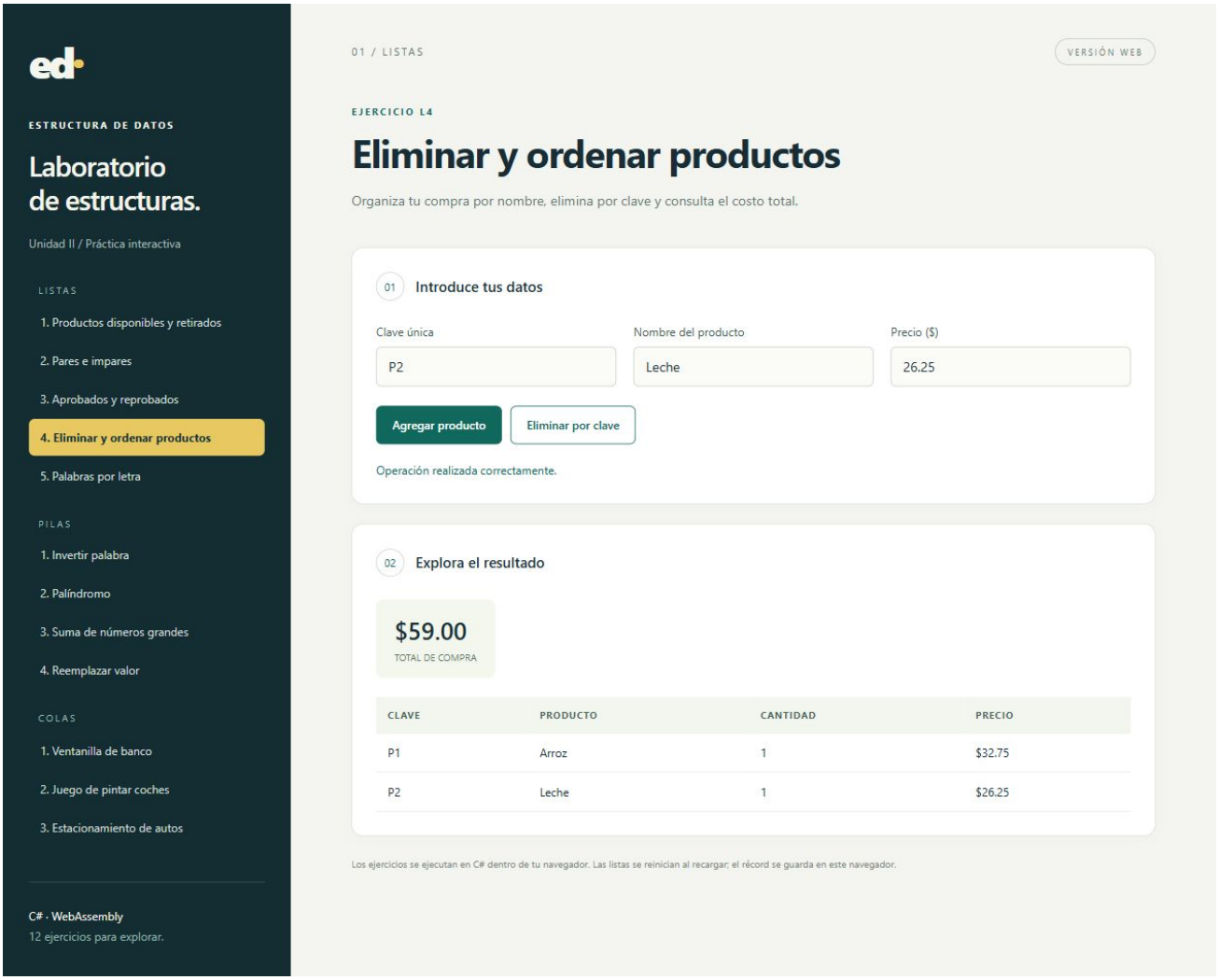
Problema: Mantener una compra con eliminación por clave y total.

Entradas: Clave única, nombre y precio decimal no negativo.

Proceso: Agrega a una lista, elimina por clave, ordena por nombre con comparación en español y suma precios usando decimal.

Salidas: Tabla ordenada y costo total.

Funcionamiento y evidencia: Agrega Arroz de 32.75 y Leche de 26.25. La imagen muestra total 59.00. Para eliminar, escribe la clave y pulsa Eliminar por clave. Se rechazan claves duplicadas y precios negativos.



L5. Palabras por letra

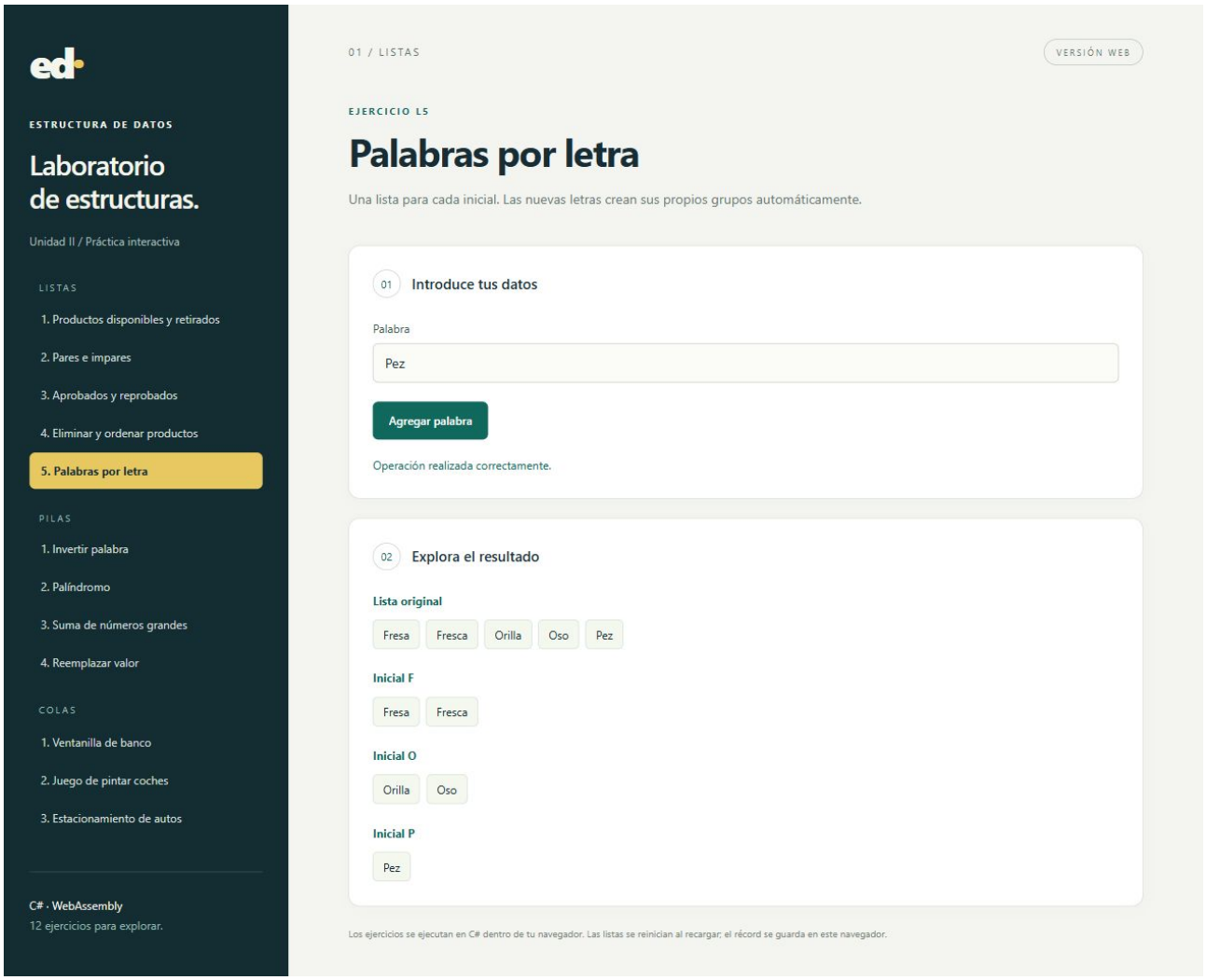
Problema: Crear listas dinámicas según la primera letra de cada palabra.

Entradas: Una palabra formada por letras en cada captura.

Proceso: Conserva la lista original y agrupa por inicial en un diccionario ordenado cuyos valores son List<string>.

Salidas: Lista original y una lista por inicial.

Funcionamiento y evidencia: Agrega Fresa, Fresca, Oso, Orilla y Pez. La captura muestra grupos F, O y P. Nuevas iniciales crean listas automáticamente; se respetan letras acentuadas.



P1. Invertir palabra

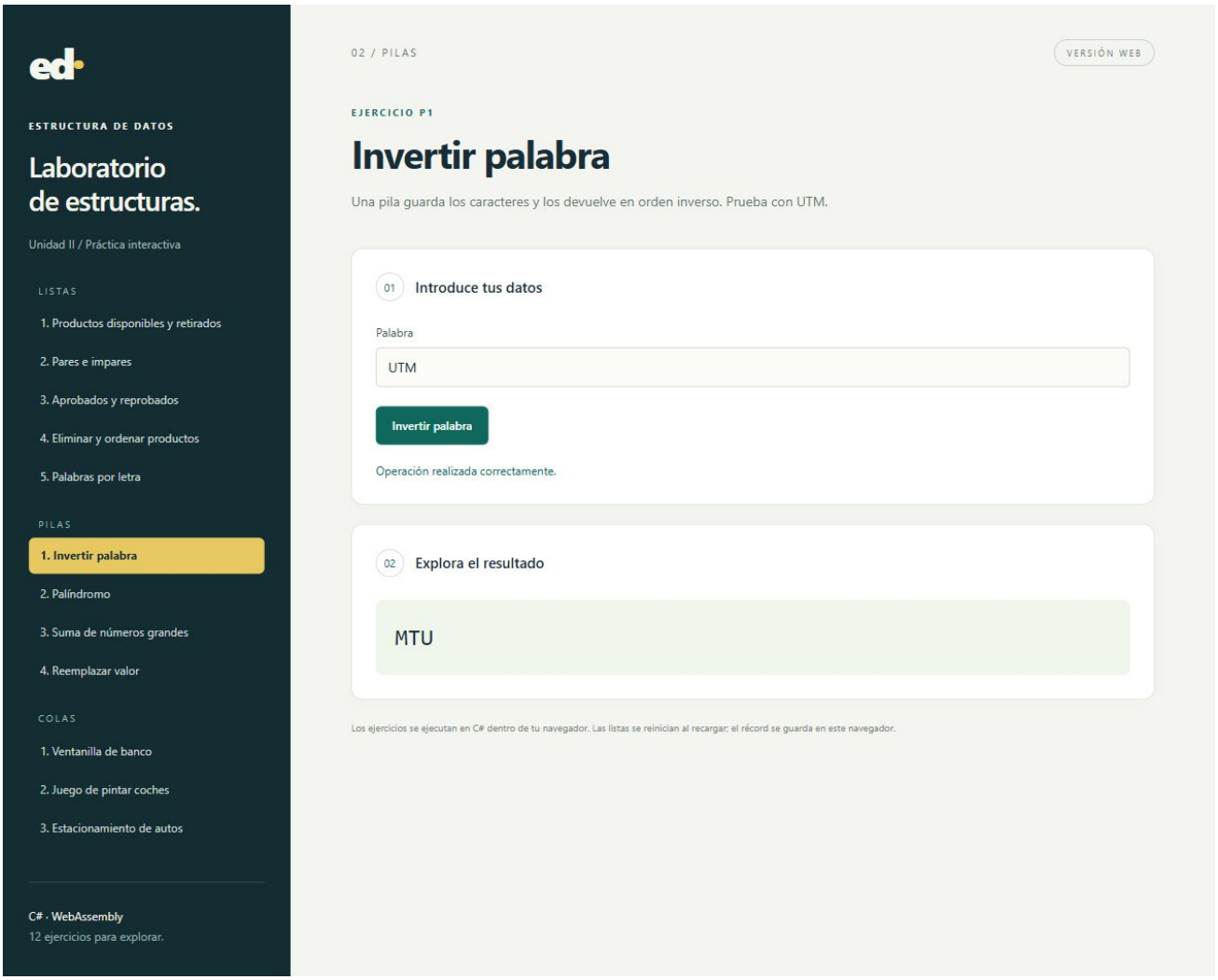
Problema: Mostrar una palabra en orden inverso mediante una pila.

Entradas: Palabra no vacía.

Proceso: Apila cada Rune del texto y desapila hasta vaciar la estructura.

Salidas: Texto invertido, preservando los caracteres Unicode completos.

Funcionamiento y evidencia: Escribe UTM y pulsa Invertir. La captura muestra MTU. El texto vacío genera un mensaje de datos faltantes.



P2. Palíndromo

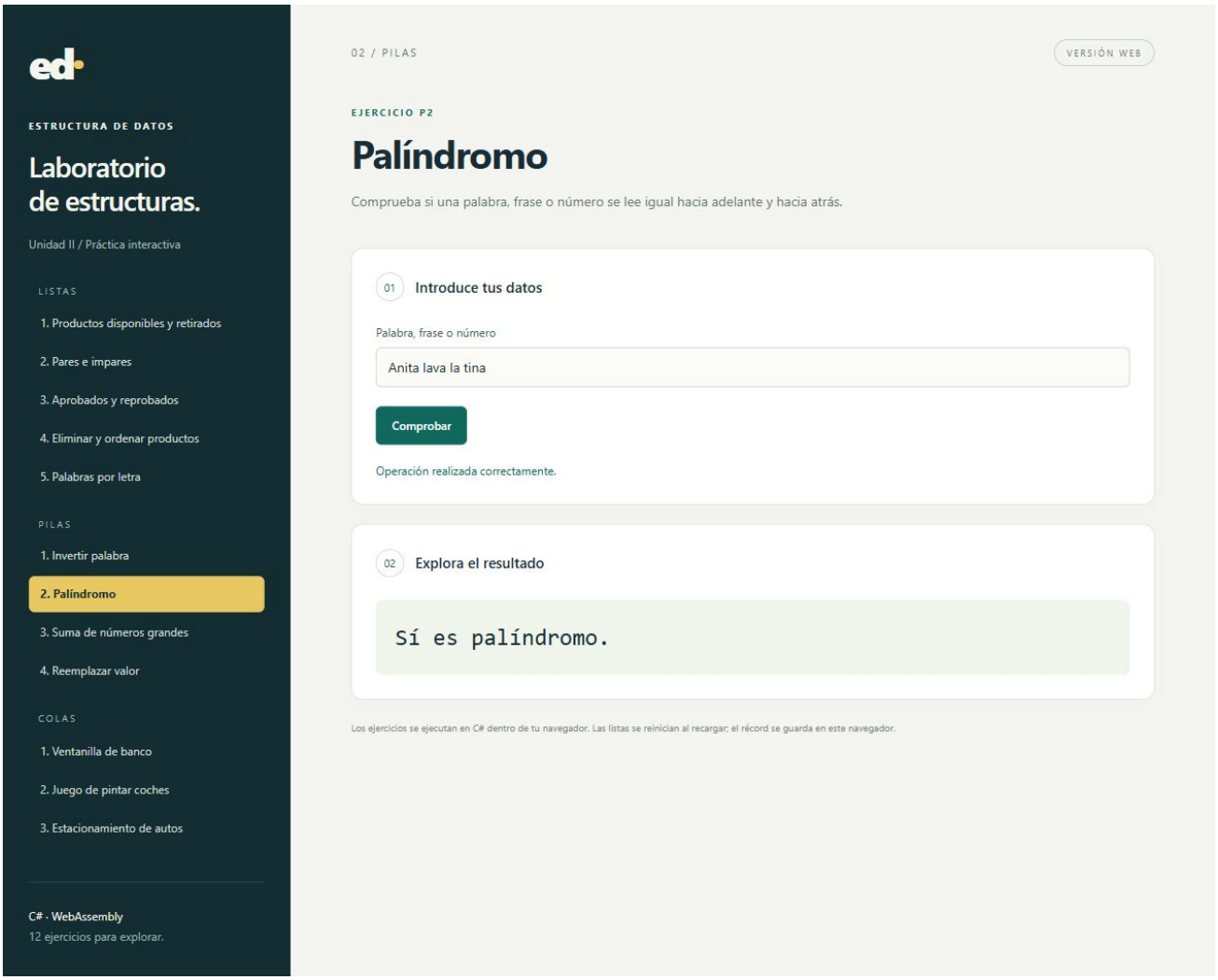
Problema: Determinar si un texto se lee igual en ambos sentidos.

Entradas: Palabra, frase o número.

Proceso: Normaliza mayúsculas y elimina espacios y signos; compara con la inversión realizada mediante pila.

Salidas: Mensaje afirmativo o negativo.

Funcionamiento y evidencia: Introduce Anita lava la tina y pulsa Comprobar. La captura muestra que es palíndromo. Se conservan acentos y se rechazan entradas compuestas únicamente por signos.



P3. Suma de números grandes

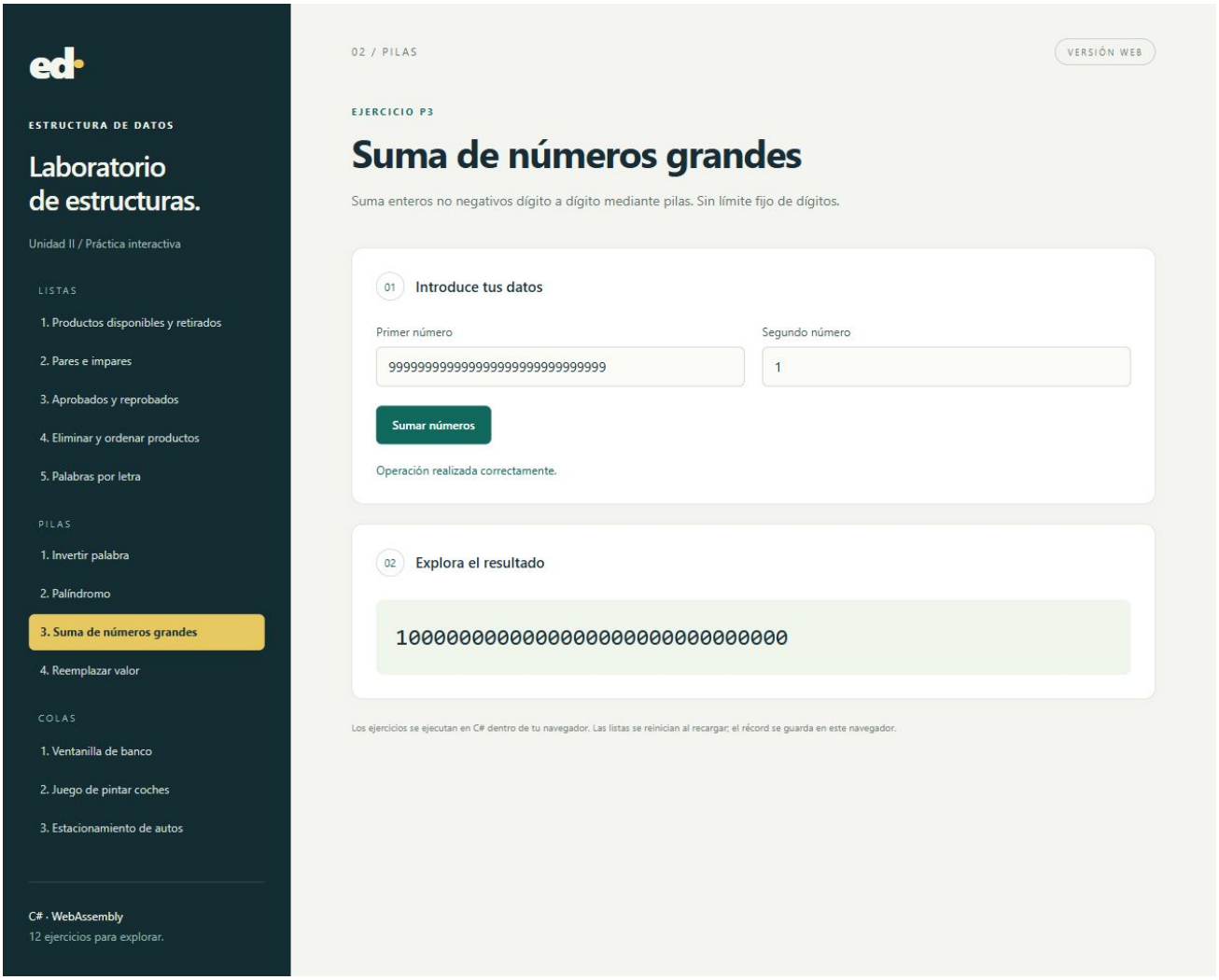
Problema: Sumar enteros que exceden los tipos numéricos ordinarios.

Entradas: Dos cadenas de dígitos, no negativas y sin separadores.

Proceso: Apila dígitos de ambos operandos, desapila de derecha a izquierda, suma con acarreo y construye el resultado con una tercera pila.

Salidas: Suma completa sin límite fijo de dígitos.

Funcionamiento y evidencia: Introduce treinta nueves y 1, y pulsa Sumar números. La imagen muestra un uno seguido de treinta ceros. Letras y signos se rechazan. BigInteger sólo se usa para comprobar las pruebas.



P4. Reemplazar valor

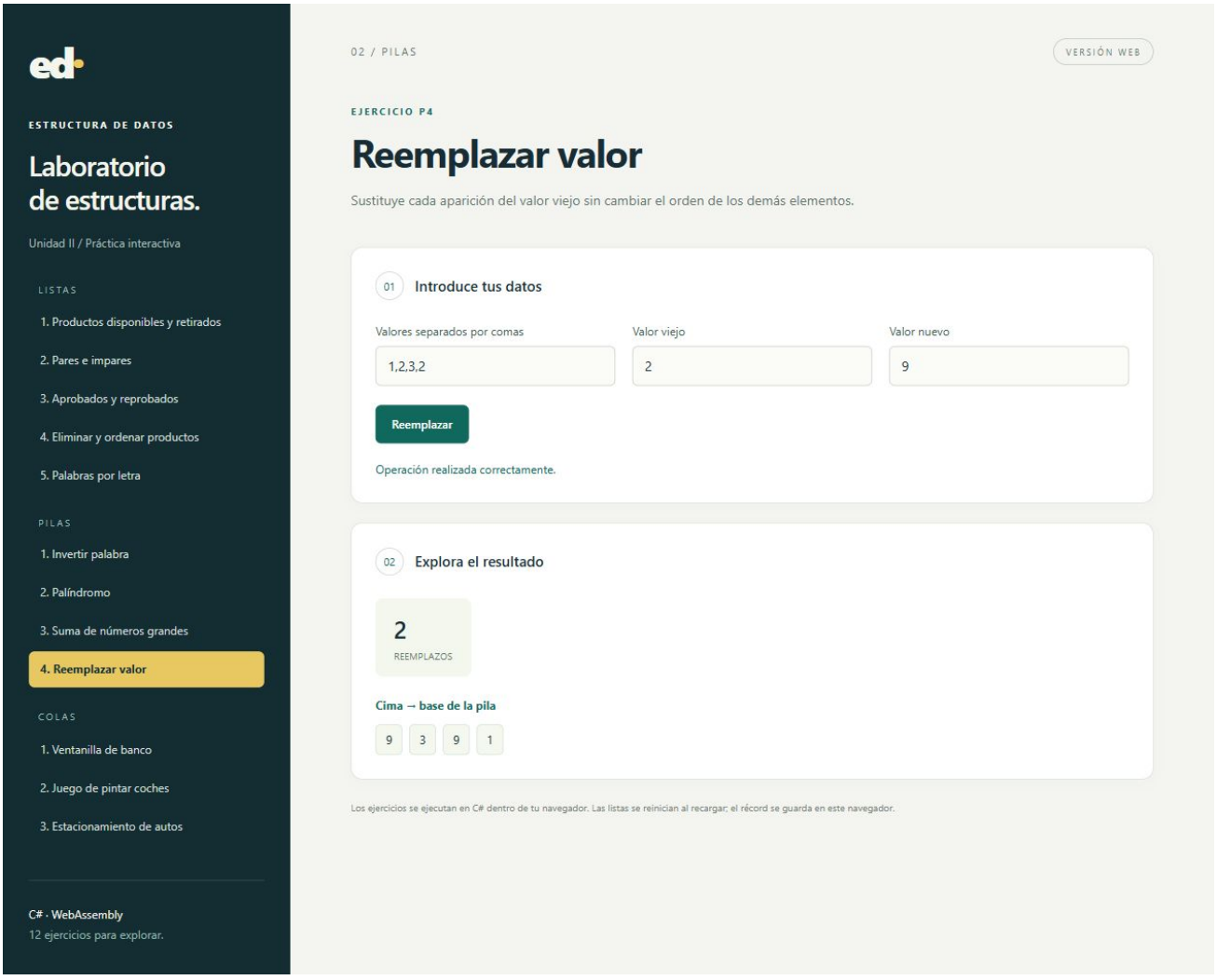
Problema: Sustituir un entero por otro en una pila sin alterar su orden.

Entradas: Enteros separados por comas, valor viejo y valor nuevo.

Proceso: Desapila hacia una pila auxiliar sustituyendo coincidencias; después restaura la pila original.

Salidas: Pila desde la cima y cantidad de reemplazos.

Funcionamiento y evidencia: Con valores 1,2,3,2, viejo 2 y nuevo 9, pulsa Reemplazar. La captura muestra 9,3,9,1 desde la cima y dos reemplazos. Se interpreta el error de redacción del enunciado como reemplazar viejo por nuevo.



C1. Ventanilla de banco

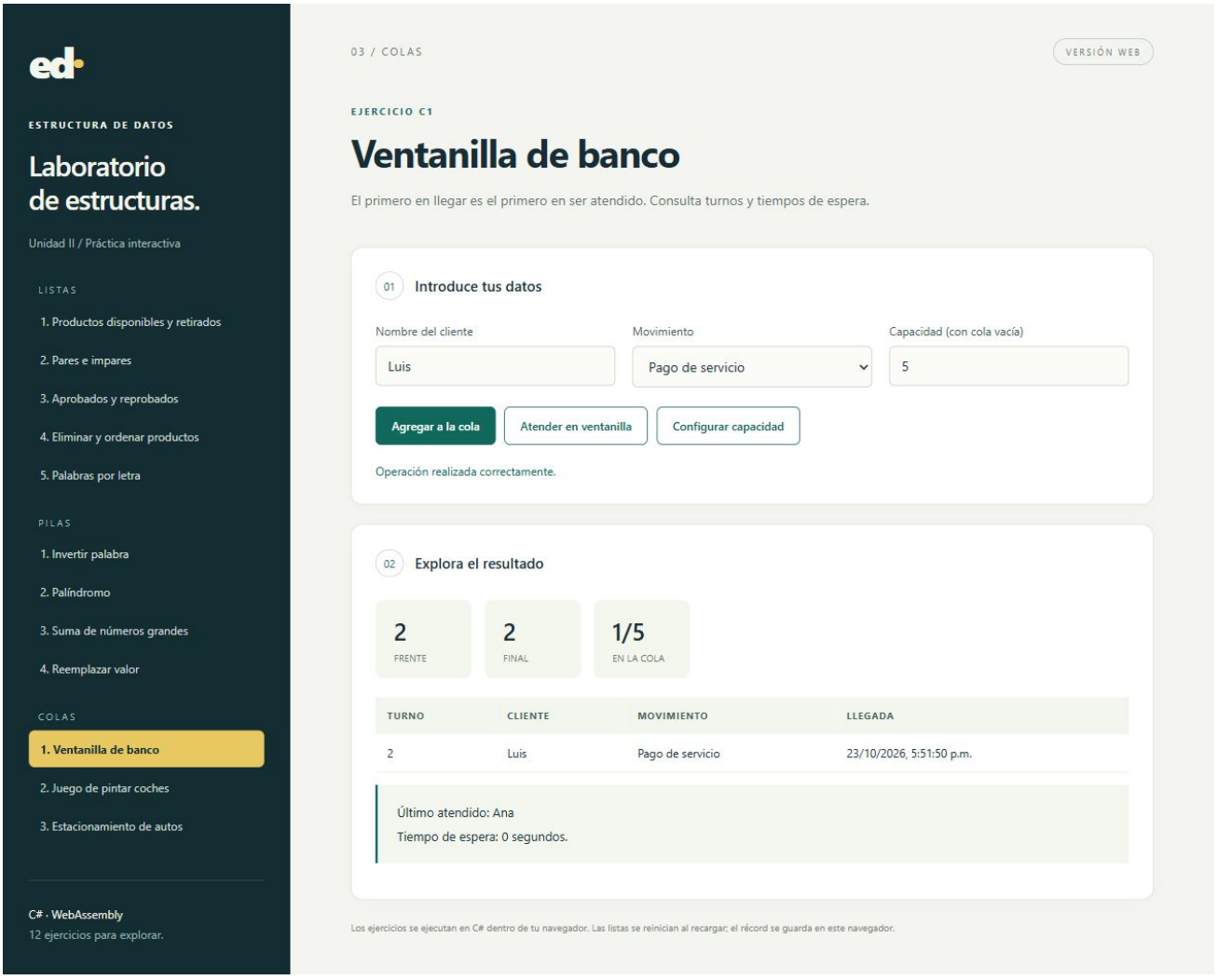
Problema: Administrar clientes mediante cola FIFO con capacidad limitada.

Entradas: Nombre, movimiento del ComboBox y capacidad configurable con cola vacía.

Proceso: Asigna turno y hora DateTime.Now; InsertaCola valida ColaLlena. Atender usa EliminaCola tras validar ColaVacia y calcula la espera.

Salidas: DataGridView actualizado, frente y final, confirmación de llegada y tiempo de espera.

Funcionamiento y evidencia: Registra Ana y Luis y atiende a Ana. La captura deja a Luis en la cola y muestra el tiempo real de espera. La capacidad inicial es cinco; se validan sobreflujo y atención con cola vacía.



C2. Juego de pintar coches

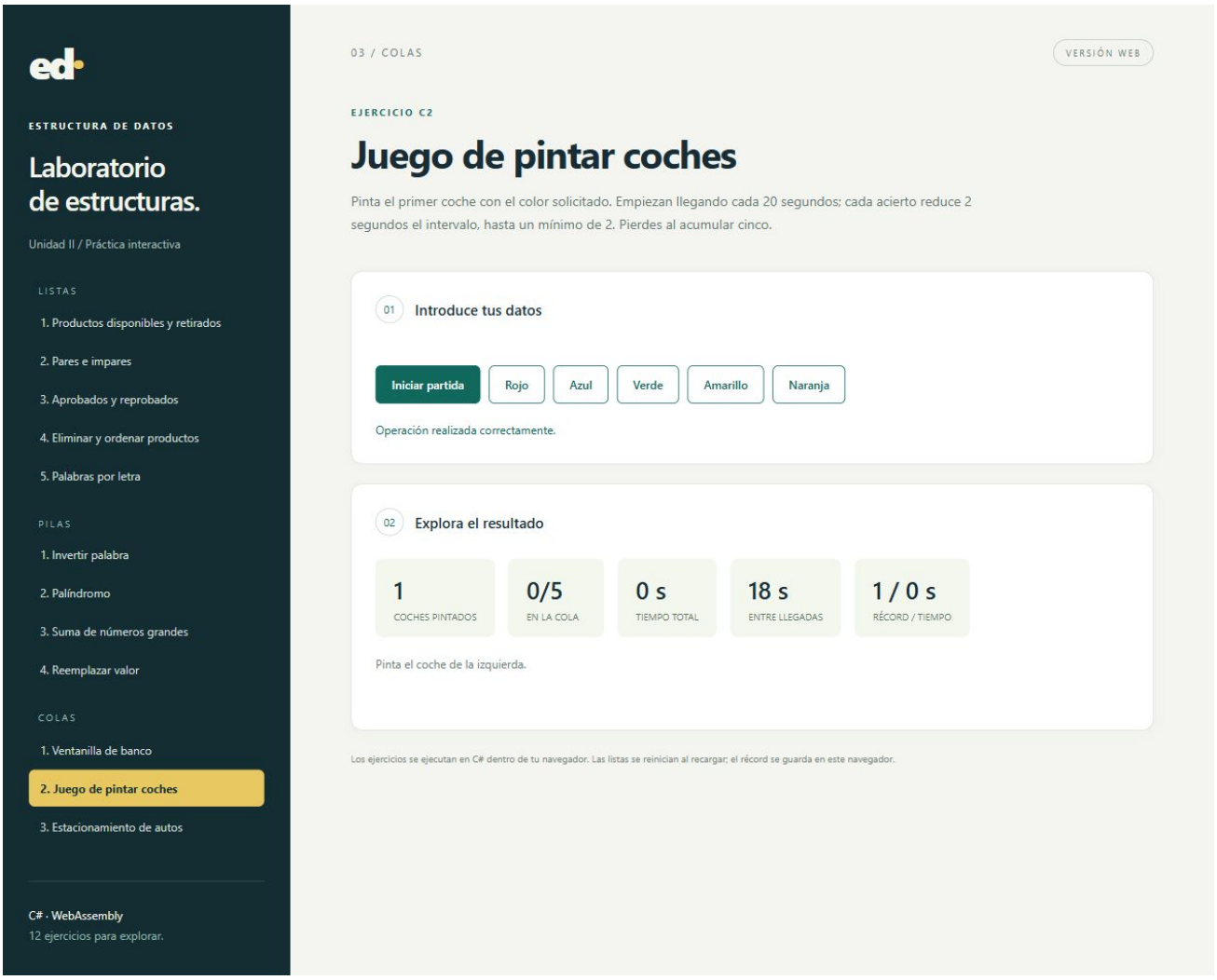
Problema: Atender coches en orden y pintarlos del color solicitado antes de que se acumulen cinco.

Entradas: Botón Iniciar partida y selección de color de la paleta de botones.

Proceso: Un reloj monótono controla llegadas y tiempo de partida. El primer coche aparece inmediatamente y el intervalo comienza en 20 segundos. Cada coche correctamente pintado reduce 2 segundos el intervalo y adelanta la siguiente llegada si corresponde, hasta un mínimo de 2 segundos. Sólo puede retirarse el frente.

Salidas: Dibujos de coches con etiqueta de color, puntuación, tiempo y récord.

Funcionamiento y evidencia: Pulsa Iniciar partida y luego el color solicitado por el coche de la izquierda. La captura muestra un acierto y el intervalo reducido de 20 a 18 segundos. Los siguientes aciertos lo reducen a 16, 14 y así hasta 2 segundos. La partida termina al acumular cinco coches. El récord y su tiempo se guardan en este navegador.



C3. Estacionamiento de autos

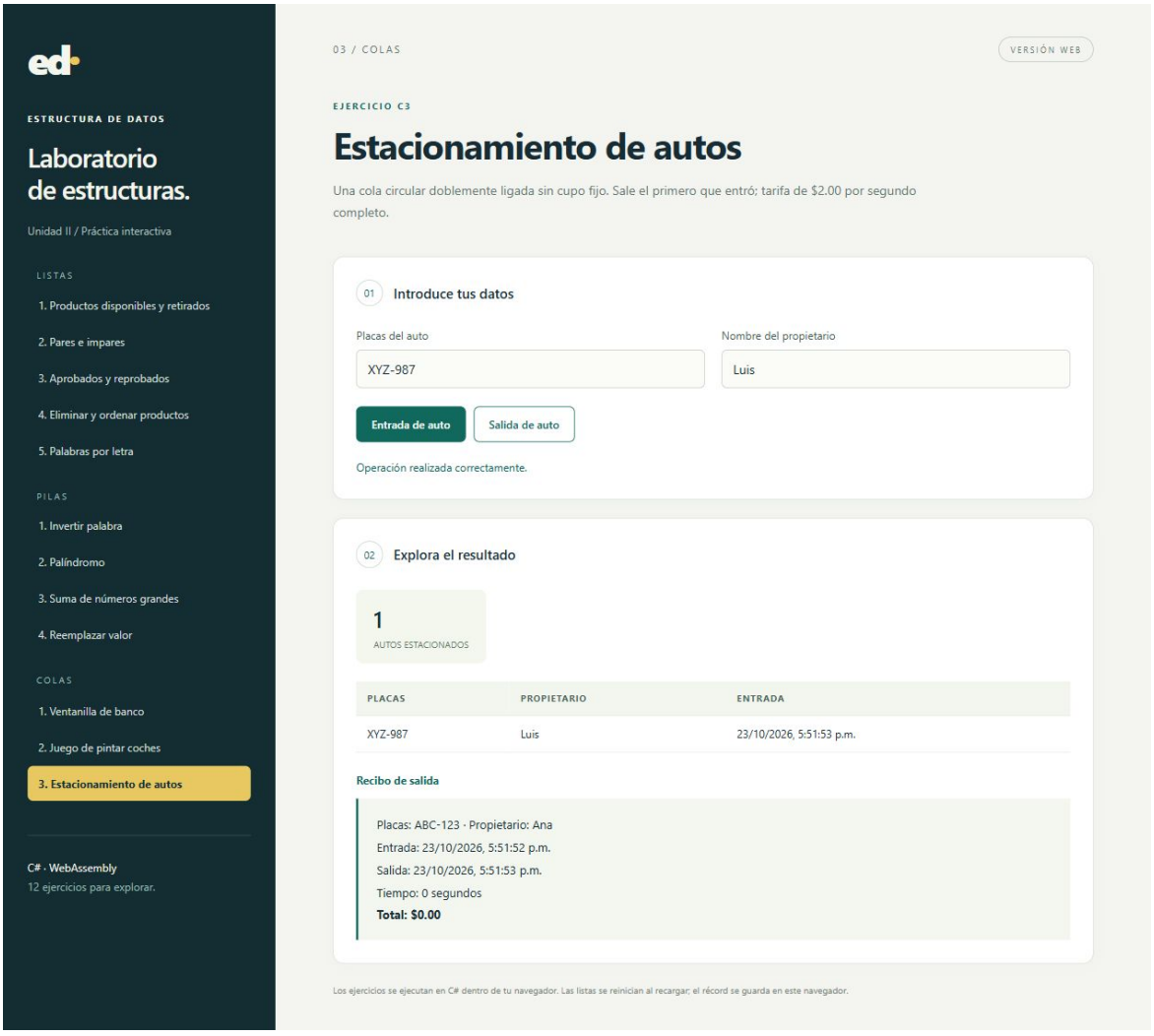
Problema: Controlar autos en un estacionamiento sin cupo fijo y cobrar dos pesos por segundo completo.

Entradas: Placas y propietario; hora de entrada y salida tomadas del sistema.

Proceso: Inserta cada auto como nodo de una cola circular doblemente ligada. Salir retira el frente, repara enlaces y calcula segundos transcurridos por 2.00.

Salidas: Tabla de autos y recibo con placas, propietario, horas, tiempo y costo.

Funcionamiento y evidencia: Registra ABC-123 y XYZ-987; pulsa Salida de auto. La captura muestra el recibo del primero y el segundo todavía en cola. Se rechazan placas repetidas y salida con estacionamiento vacío.



Pruebas y decisiones

Se verificaron los doce ejercicios por sus controles en la versión C# WebAssembly y el cambio de dificultad después del primer acierto. La variante ASP.NET además tiene pruebas HTTP de validación y aislamiento de sesiones.

pruebas-publica.txt

OK | L1: controles y resultado C# WebAssembly

OK | L2: controles y resultado C# WebAssembly

OK | L3: controles y resultado C# WebAssembly

OK | L4: controles y resultado C# WebAssembly

OK | L5: controles y resultado C# WebAssembly

OK | P1: controles y resultado C# WebAssembly

OK | P2: controles y resultado C# WebAssembly

OK | P3: controles y resultado C# WebAssembly

OK | P4: controles y resultado C# WebAssembly

OK | C1: controles y resultado C# WebAssembly

OK | C2: controles y resultado C# WebAssembly

OK | Dificultad: el primer acierto reduce 20 a 18 segundos

OK | C3: controles y resultado C# WebAssembly

TOTAL: 12 ejercicios web verificados en navegador; dificultad progresiva comprobada.

pruebas-web.txt

OK | Página web disponible

OK | L1: escanear y retirar

OK | L2: clasificación completa

OK | L3: límite de aprobación

OK | L4: orden y total decimal

OK | L4: eliminar por clave

OK | L5: grupos por letra

OK | P1: invertir palabra

OK | P2: palíndromo

OK | P3: acarreo de mil dígitos

OK | P4: reemplazar conservando el orden

OK | C1: rechazar sobreflujo

OK | C1: atención FIFO y cola vacía

OK | C2: partida y coche inicial

OK | C2: pintar el primer coche

OK | C2: el primer acierto reduce el intervalo a 18 segundos

OK | C3: salida FIFO y recibo

OK | Validación: nota fuera de rango

OK | Validación: suma inválida

OK | Validación: placas repetidas

OK | Sesiones: datos separados por navegador

OK | Solicitudes: cabecera requerida

TOTAL: 22 verificaciones web correctas. Sin fallos.

En la versión pública los datos de los ejercicios permanecen al navegar y se reinician al recargar. El récord del juego se conserva en el almacenamiento local. La variante ASP.NET usa sesiones del servidor, con expiración por inactividad o reinicio. La tarifa de estacionamiento es de $2.00 por segundo completo; la salida es FIFO.

Se resuelven los errores de redacción del enunciado como: mostrar aprobados y reprobados, y sustituir el valor viejo por el nuevo manteniendo el orden de la pila. El cambio de dificultad por cada coche pintado fue solicitado expresamente por el equipo.